

Wassat 0.1: Formula-Directed SAT Solving in Tungsten

Erik Peterson
Independent Researcher

ABSTRACT

Abstract- Wassat is an original SAT solver written in Tungsten. An input-directed policy combines a flat-array conflict-driven clause-learning core with stochastic local search, bounded preprocessing, and strict structural recognizers. The sequential configuration emits DRAT certificates; the parallel configuration races diversified Wassat instances and model-only specialists. Every satisfying assignment is reconstructed and checked against the original CNF. This document describes revision 29142c9665a4+dirty, comprising 24,025 lines of Tungsten solver source.

I. CORE SOLVER

Wassat implements two-watched-literal Boolean constraint propagation, first-UIP conflict analysis, learned-clause minimization, variable and clause activity, LBD-guided database reduction, chronological and non-chronological backtracking, phase saving, rephasing, and restart schedules. Its hot state is held in typed flat arrays. The input parser is strict DIMACS CNF: it validates headers, counts, terminators, integer ranges, and literal bounds before any unchecked solver access.

The decision engine can use EVSIDS or a VMTF queue. Formula statistics - variables, clauses, literals, units, binary and ternary clauses, maximum width, and estimated memory - drive the automatic branching, preprocessing, scout, local-search, and portfolio policy.

Preprocessing includes bounded variable elimination, subsumption, strengthening, equivalence handling, probing, and model-reconstruction records. Transformations used by the certified sequential configuration carry their certificate obligations into the emitted proof. More aggressive transformations and structure-specific UNSAT decisions remain confined to the trusted parallel configuration until their proof emission is implemented; replay-checked model-only SAT lanes may also answer in the sequential configuration.

II. SAT AND STRUCTURED SEARCH

The CCAnr-family local-search engine maintains weighted unsatisfied clauses, configuration checking, incremental break scores, and its best assignment. Failure never implies unsatisfiability. Near solutions can seed CDCL phases; a win is accepted only after checking the model against the original CNF.

Bounded recognizers target exact structures that are expensive for generic CDCL but cheap in their native representation. They cover local cores; independent, covering, coloring, directed-kernel, and Latin constraints; multiplier and fixed-width Fermat circuits; bounded sum-of-three-cubes and Minimum Disagreement Parity encodings; synchronizing automata; edge-matching grids; sliding puzzles; Stedman and Erin triples; distance-pruned knight tours; the published Hantsche--Wendt group-ring unit; and selected ternary-affine encodings. The knight lane recovers unlabeled position/square incidence and transitions before generating a tour; the group-ring lane expands Gardam's polynomials and multiplication law rather than storing a model. Every recognizer has a strict structural gate. SAT answers pass the original-CNF model checker; UNSAT is used only where the recognized subset or exact search supplies a sound refutation.

A bounded GF(2) side arm groups complete parity encodings and narrowly recognized near-complete rows whose missing ternary clause is implied by a uniquely owned binary subclause. It performs dense-coordinate Gaussian elimination, then enumerates only a bounded residual affine space. An inconsistent recognized subset refutes the formula. Otherwise back-substitution supplies only candidates; Wassat reports SAT only after one satisfies every original clause, so mixed formulas remain sound.

An exact circuit recognizer accepts a topologically ordered 32-bit xorshift/fold program only when variables 1--32 are its sole inputs, later variables have the expected gate definitions, and 32 units pin the accumulator. Native workers partition the 4,294,967,296-point preimage domain. A hit is replayed and checked against the original CNF; exhaustion publishes no UNSAT claim.

III. SEQUENTIAL AND PARALLEL CONFIGURATIONS

The Main-style sequential configuration writes ASCII DRAT to `proof.out` for UNSAT; checked model-only SAT paths may finish before CDCL. Wassat includes an independent WRAT checker. Host-local qualification uses `drat-trim` and, when available, the competition's DRAT-to-LPR pipeline and formally verified `cake_lpr`; Linux, NHR, and AWS acceptance remain pending.

The Parallel configuration runs a portfolio composed entirely of Wassat code. Raw CDCL arms differ in branching, phases, chronological backtracking, shrinking, subsumption, restart behavior, and other internally selected search axes. Low-LBD clauses move through a bounded lock-free publication ring. SLS, preprocessing, and exact recognizers provide genuinely different solving methodologies beside CDCL. A first decisive result cooperatively cancels the remaining work. Parallel SAT output uses one or more `v` lines of at most 4096 characters, terminated by `0`.

IV. CORRECTNESS AND EVALUATION

Wassat treats model and proof checking as separate trust boundaries. SAT models are reconstructed after preprocessing and checked against the original formula. Proofs are flushed to a temporary sibling and atomically renamed only after complete UNSAT; interruption may leave that temporary file but never exposes it as the final certificate. Exit codes are 10 for SAT, 20 for UNSAT, 0 for UNKNOWN, and 1 for input or usage errors.

The maintained gate includes language-level unit specifications, randomized SAT/UNSAT differential tests, independent model checks, independently replayed raw UNSAT proofs, proof-format regressions, and competition-output checks. Competition-scale results are reported with the exact corpus, timeout, hardware, solver revision, and PAR-2 policy so tuned development subsets are not presented as blind evaluation.

V. CODE BASE AND AI DISCLOSURE

Wassat 0.1 is an original implementation with zero inherited solver lines, not a patch series or fork. The submission pins Tungsten revision 29142c9665a4+dirty and contains 24,025 lines across its entry point and libraries. It carries the same-revision compiler and runtime for bootstrap without dependency downloads; NHR and AWS acceptance remain pending. Literature and permissively licensed public generators informed independently written mechanisms; third-party solver code is not linked. Required MDP attribution is included.

OpenAI Codex was used extensively for implementation, review, tests, profiling, competitor-source study, and parameter evaluation. Human and AI edits were repeatedly revised together, so we conservatively classify 100 percent of the shipped source as AI-generated or AI-assisted. Erik Peterson selected the objectives, reviewed and accepted the work, and is responsible for the submission.

AI-assisted tuning covered routing, budgets, portfolio axes, search schedules, preprocessing, local search, sharing, and recognizer gates. Offline tooling can train candidate routers from paired same-binary interventions with family-disjoint PAR-2 and regret scoring, but no learned selector is shipped. Its feature ABI includes clause-width and polarity statistics, an exact-one sketch, binary occurrence-graph statistics, and variable-occurrence concentration. A future selector requires a held-out win; rejected interventions default off.

REFERENCES

- [1] J. P. Marques-Silva and K. A. Sakallah, "GRASP: A search algorithm for propositional satisfiability," IEEE Transactions on Computers, 1999.
- [2] N. Een and N. Sorensson, "An extensible SAT-solver," SAT 2003.
- [3] G. Audemard and L. Simon, "Predicting learnt clauses quality in modern SAT solvers," IJCAI 2009.
- [4] S. Cai and K. Su, "Configuration checking with aspiration in local search for SAT," AAAI 2013.
- [5] M. J. H. Heule, W. A. Hunt, and N. Wetzler, "Trimming while checking clausal proofs," FMCAD 2013.
- [6] SAT Competition 2026, "Rules, tracks, and output requirements," <https://satcompetition.github.io/2026/>.
- [7] M. Gardam, "A counterexample to the unit conjecture for group rings," Annals of Mathematics, 2021.